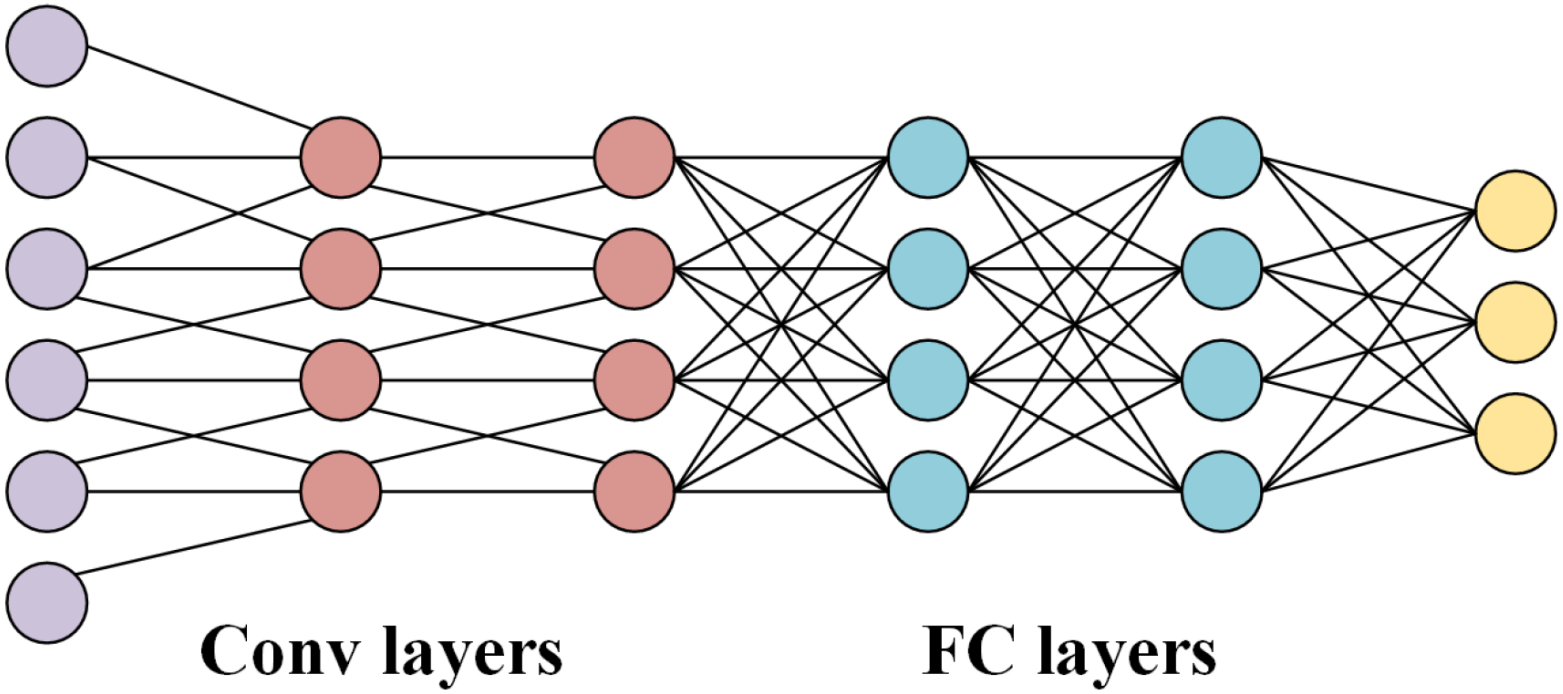
Fundamentals of Deep Learning and Programming

Lecture 5: Convolutional Neural Network (CNN)

Fall Semester 2026

Adjunct Professor: Donghoon Kang

Convolution



Convolutional Neural Network (CNN)

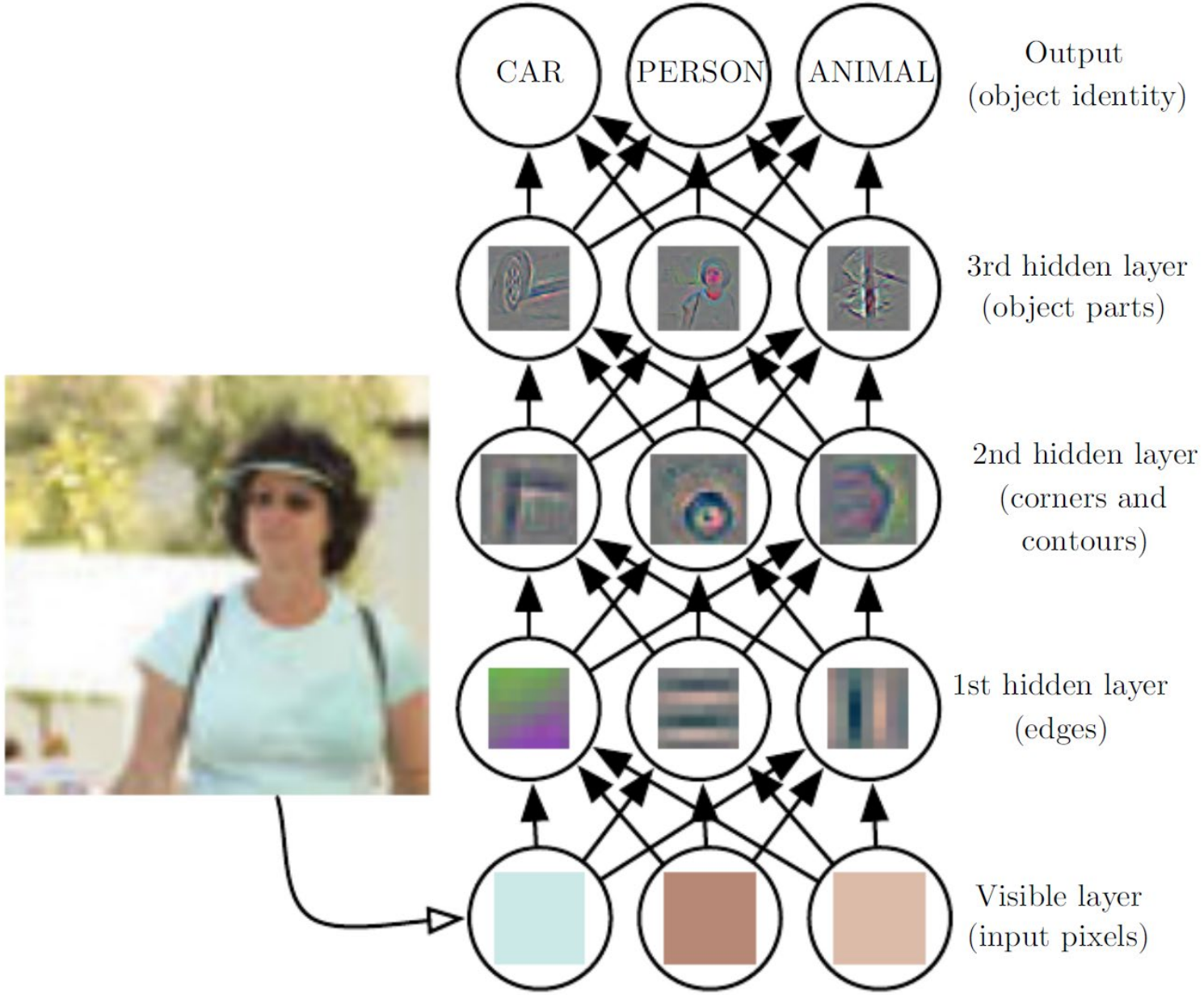
Locality

- Convolutional filters in CNNs look at local regions (receptive fields) rather than the entire image at once.
- Instead of processing the whole image, they first capture local features (e.g., edges, corners, color changes) within small patches such as 3×3 or 5×5.
- By stacking multiple layers, CNNs gradually build up from low-level features (lines, edges) to high-level features (faces, objects).

Convolutional Neural Network (CNN)

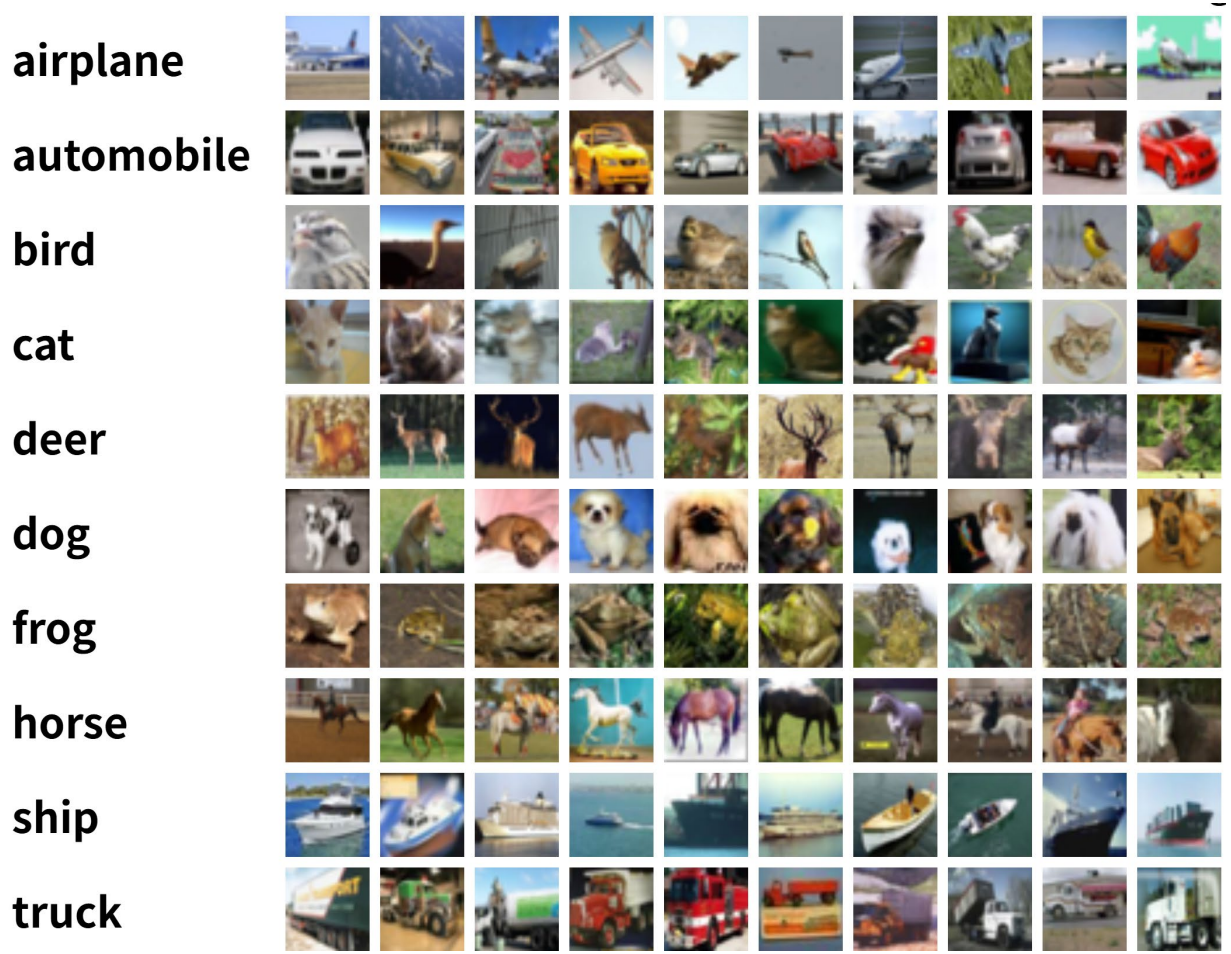
Translation Invariance

- CNNs can recognize the same feature even if the object appears at a slightly different location in the image.
- This property comes from two main mechanisms:
- Convolution + Shared Weights
- The same filter (kernel) slides across the entire image, so a feature can be detected regardless of its position.
- Pooling (especially Max Pooling)
- Pooling summarizes nearby pixels, making the network more robust to small shifts (translations).
- ⚠ However, CNNs do not achieve perfect translation invariance:
  They are robust to small shifts due to pooling and stride, but large translations can still cause errors.


CAR
PERSON
ANIMAL
Output
(object identity)
3rd hidden layer
(object parts)
2nd hidden layer
(corners and
contours)
1st hidden layer
(edges)
Visible layer
(input pixels)

CIFAR-10 Dataset

The CIFAR-10 dataset consists of 60,000 **32 x 32** colour images in **10** classes, with 6,000 images per class. There are 50,000 training images and 10,000 test images.



Download CIFAR-10 dataset manually

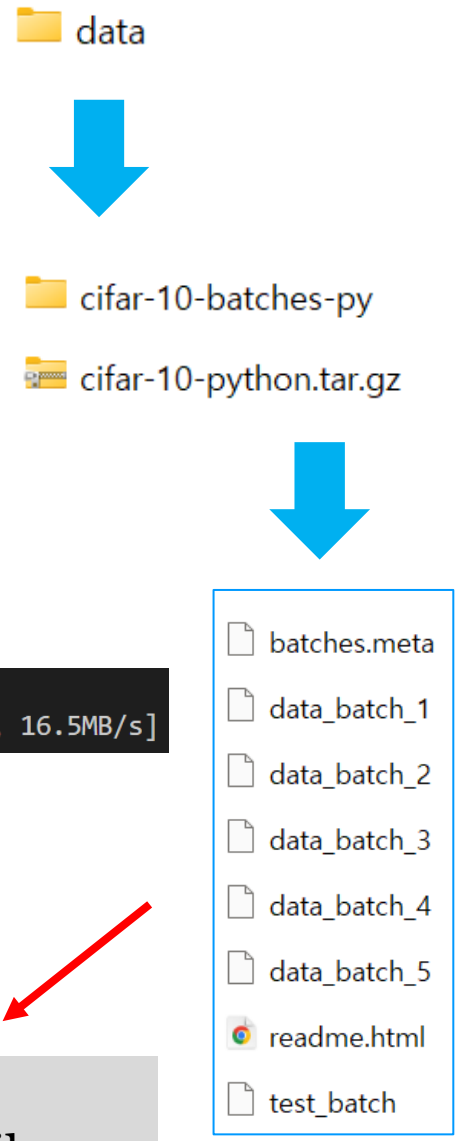
```python
# download_cifar10.py
import torchvision.datasets as datasets

# Download CIFAR-10 (train and test) into the "data" folder
train_dataset = datasets.CIFAR10(root="./data", train=True, download=True)
test_dataset  = datasets.CIFAR10(root="./data", train=False, download=True)

print("CIFAR-10 dataset has been downloaded !")
print(f"Training set size: {len(train_dataset)}")
print(f"Test set size: {len(test_dataset)}")
```

download_cifar10.py

```
Downloading https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz to ./data\cifar-10-python.tar.gz
100%| 170M/170M [00:10<00:00, 16.5MB/s]
Extracting ./data\cifar-10-python.tar.gz to ./data
Files already downloaded and verified
CIFAR-10 dataset has been downloaded !
Training set size: 50000
Test set size: 10000
```



You don't see any .jpg or .png files
because CIFAR-10 images are not stored as actual image files.

What does “binary file” mean ?

A **binary file** is a file that stores data **in bytes (0s and 1s)** — **not** as human-readable text.

For Example:

.txt file might look like this:

```
Hello world
```

But a **binary file** might look like this if opened in a text editor:

```
¤íÛ@ÉÝÀÿ…
```

Binary files are optimized for computers, not for humans.
They store numbers, arrays, and images directly **in byte format**.

Convolution (1)

Example 1

0.1	0.3	0.4
0.7	0.6	0.5
0.2	0.1	0.3

Input feature map

⊗

a	b
c	d

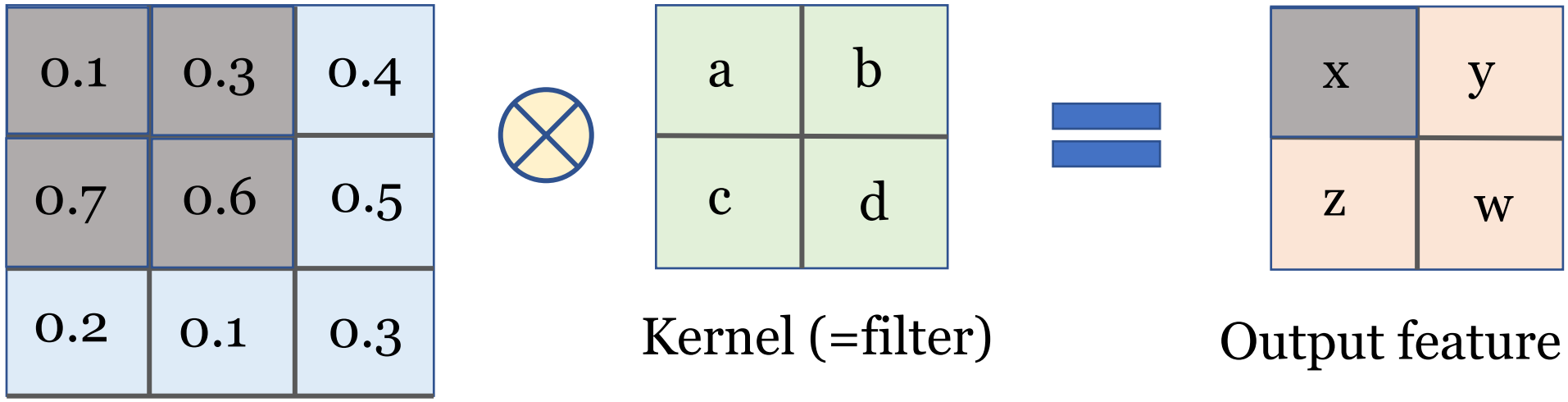
Kernel (=filter)

=



Output feature map

Convolution (1)



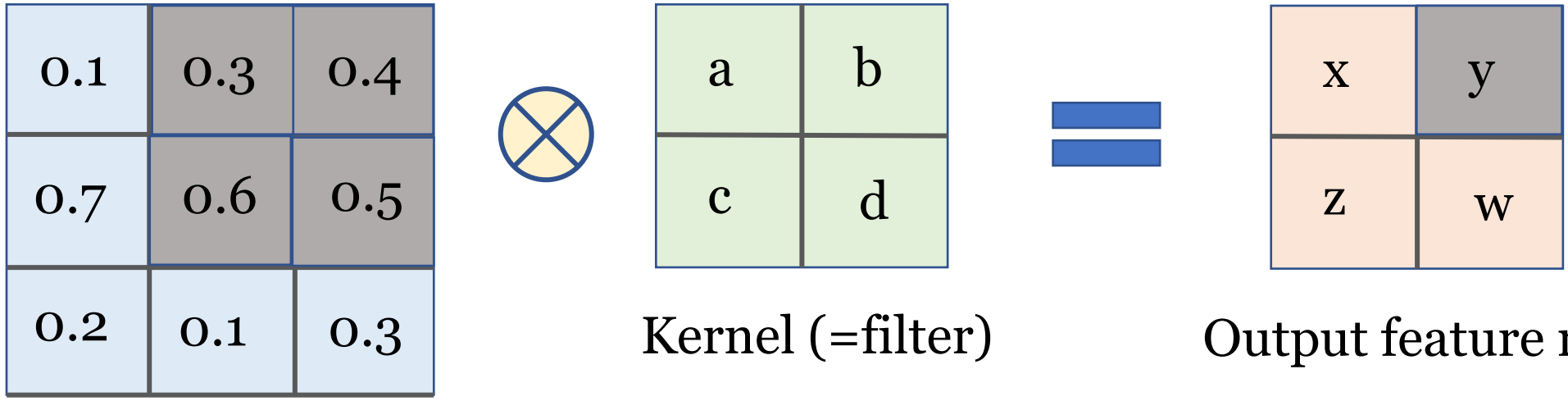
Input feature map

Kernel (=filter)

Output feature map

$$x = 0.1a + 0.3b + 0.7c + 0.6d$$

Convolution (1)



Input feature map

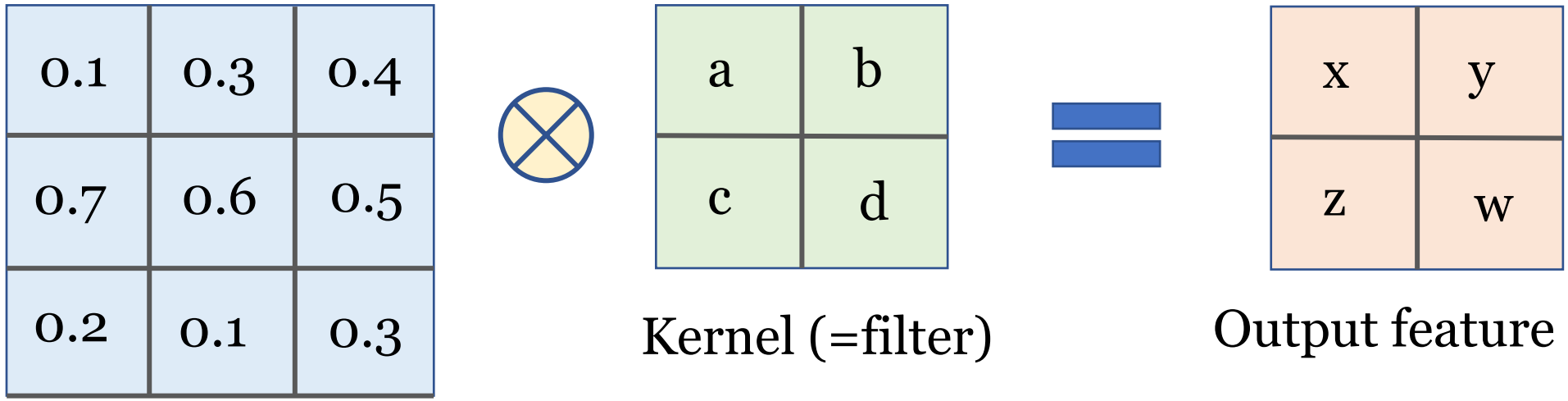
Kernel (=filter)

Output feature map

$$y = 0.3a + 0.4b + 0.6c + 0.5d$$

Stride = 1

Convolution (1)



Input feature map

Kernel (=filter)

Output feature map

Stride = 1

$$x = 0.1a + 0.3b + 0.7c + 0.6d$$

$$y = 0.3a + 0.4b + 0.6c + 0.5d$$

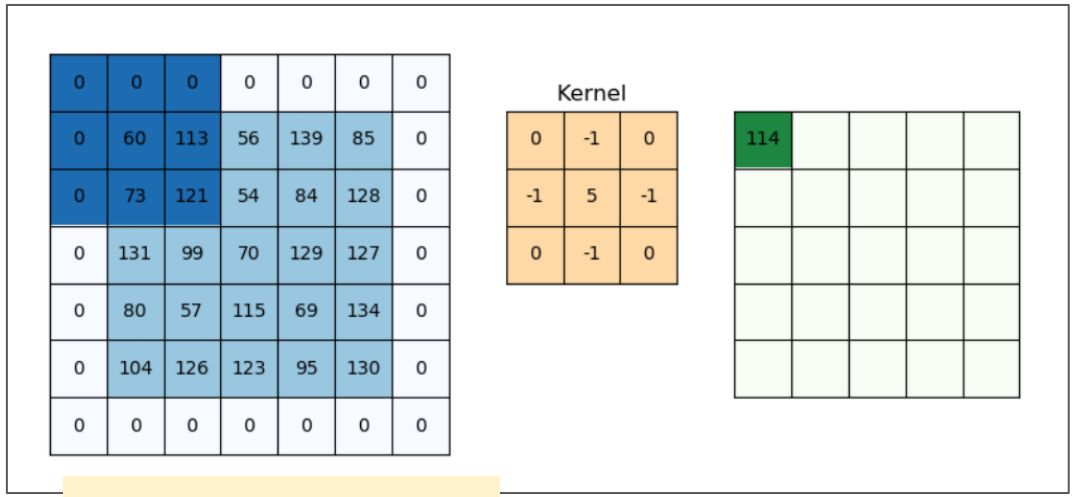
$$z = 0.7a + 0.6b + 0.2c + 0.1d$$

$$w = 0.6a + 0.5b + 0.1c + 0.3d$$

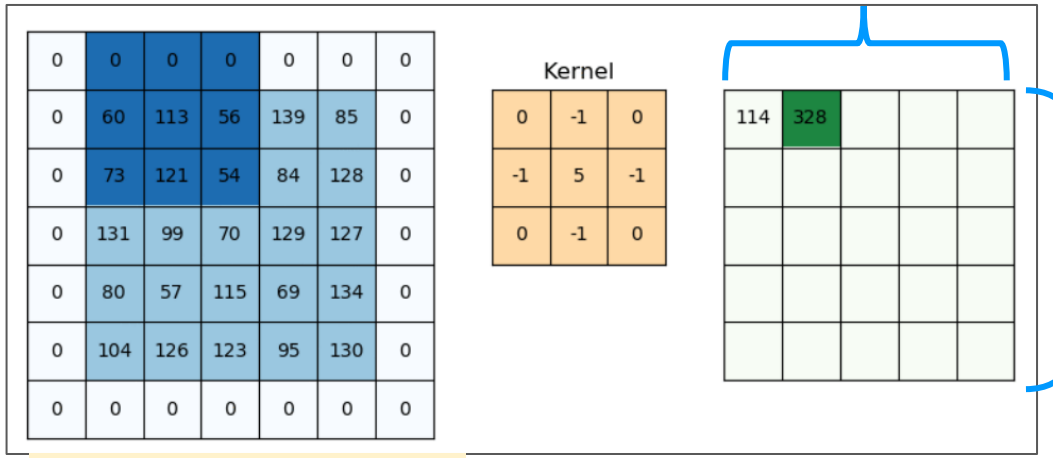
Zero Padding

Example 2

Stride = 1



Zero Padding



Zero Padding

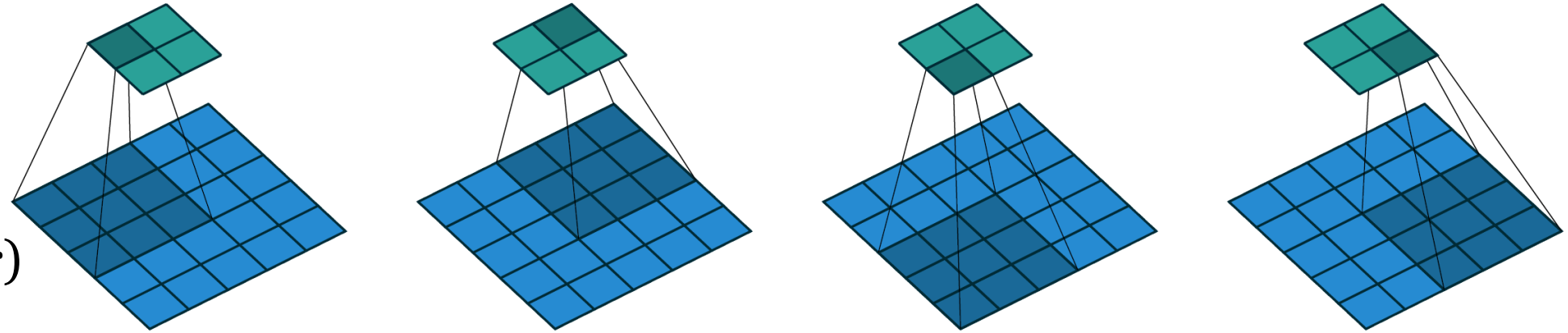
$$o = \left\lfloor \frac{i + 2p - k}{s} \right\rfloor + 1.$$

o: output size
i : input size
p: padding size
k: kernel size
s: stride

$$o = \left\lfloor \frac{5 + 2 - 3}{1} \right\rfloor + 1 = 4 + 1 = 5$$

Example 3

Output feature map



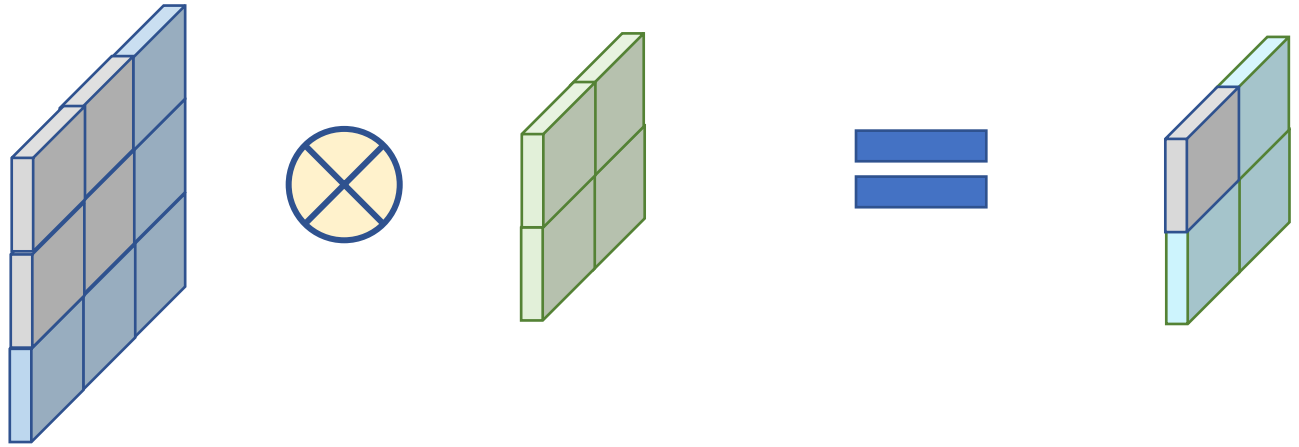
Kernel (=filter)

Input feature map

Convolving a 3 × 3 kernel over a 5 × 5 input using 2 × 2 strides (no padding)

$$o = \frac{5 + 0 - 3}{2} + 1 = 1 + 1 = 2$$

single kernel



1 × 3 × 3

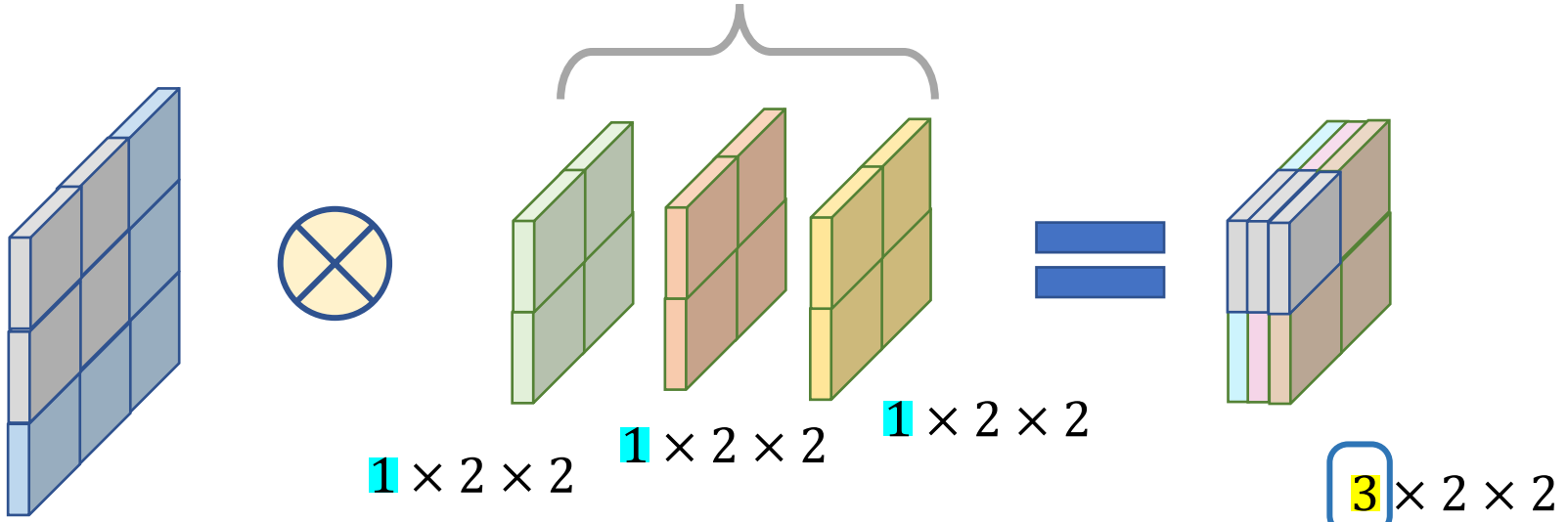
(channel, width, height)

1 × 2 × 2

Input feature map Kernel (=filter) Output feature map

Example 3

multiple kernels



1 × 3 × 3

(channel, width, height)

Input feature map

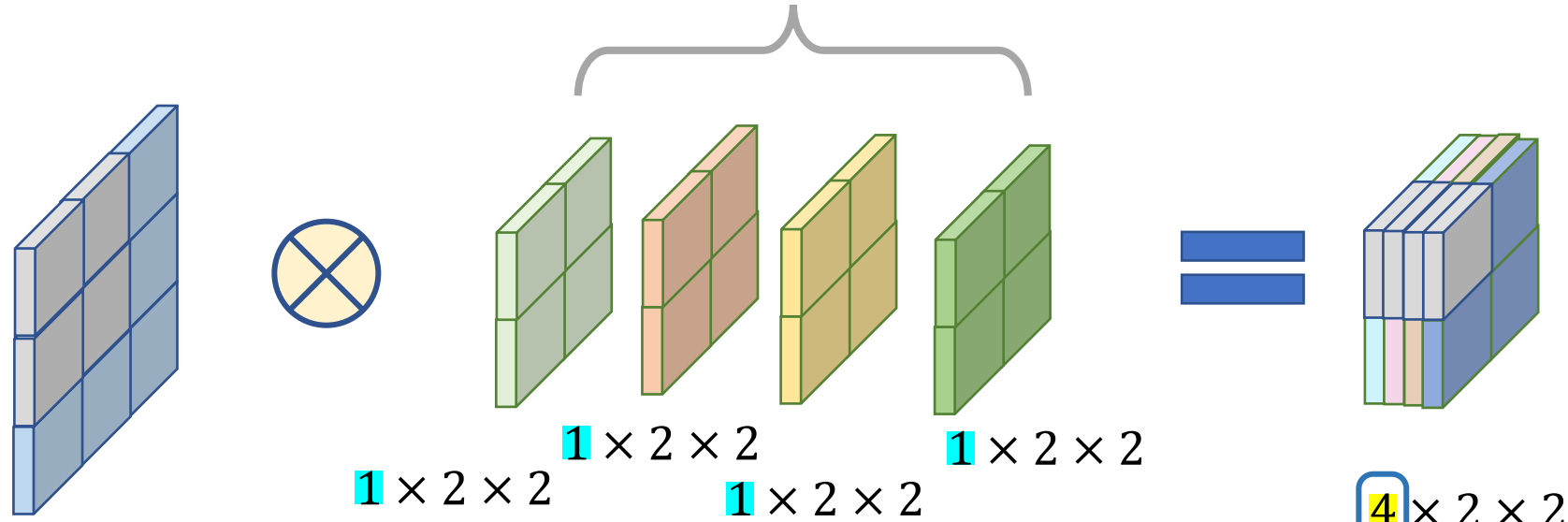
3 Kernels (=filters)

Output feature map

Convolution (4)

Example 4

multiple kernels



1 × 3 × 3

(channel, width, height)

Input image

4 Kernels (=filters)

Feature map

Convolution layers in PyTorch

- The `torch.nn` module includes convolution layers for 1D, 2D, and 3D data:

Layer	Typical Input	Use Case	Example **Input Shape**
`nn.Conv1d`	1D signals	Audio, text (sequences), or 1D feature maps	[batch, channels, length]
`nn.Conv2d`	2D images	Image processing (CNNs)	[batch, channels, height, width]
`nn.Conv3d`	3D volumes	Video, medical imaging (e.g., MRI)	[batch, channels, depth, height, width]

nn.Conv1d in PyTorch

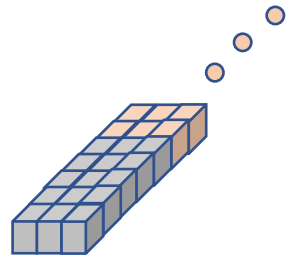
```python
import torch
import torch.nn as nn

# 1D Conv ================================================
x = torch.randn(1, 3, 100)  # batch=1, channels=3, length=100

conv1d = nn.Conv1d(in_channels=3, out_channels=6, kernel_size=5, stride=1, padding=2)
y = conv1d(x)
print(y.shape)  # [1, 6, 100]
```

simple_conv.py

```
torch.Size([1, 6, 100])
```



$1 \times 3 \times 100$

(batch, channel, length)

```python
torch.nn.Conv1d(
    in_channels,      # Number of input channels
    out_channels,     # Number of output channels (number of filters)
    kernel_size,      # Size of the 1D convolution filter
    stride=1,         # Step size of the convolution
    padding=0,        # Zero-padding added to both sides of the input
    dilation=1,       # Spacing between elements in the kernel
    groups=1,         # Controls connections between input and output channels
    bias=True,        # Whether to include a bias term
    padding_mode='zeros'  # Padding mode ('zeros', 'reflect', etc.)
)
```

$$o = \left\lfloor \frac{i + 2p - k}{s} \right\rfloor + 1.$$

o: output size
i : input size
p: padding size
k: kernel size
s: stride

$$o = \frac{100 + 2 \times 2 - 5}{1} + 1 = 100$$

What is **dilation** in CNN ?

- Dilation controls the spacing between elements (weights) inside the convolution kernel.
- Normally, in a standard convolution (dilation=1), the filter slides over consecutive elements of the input.
- When you set dilation > 1, the filter "skips" elements — it creates a wider receptive field without increasing the number of parameters.

Example 1D Kernel size : 3

Dilation = 1:



Dilation = 2:



Dilation = 3:



Let's say we have a 1D kernel of size 3:

- dilation=1 → use input positions [t, t+1, t+2]
- dilation=2 → use [t, t+2, t+4]
- dilation=3 → use [t, t+3, t+6]

What is **receptive field** in 1D CNN ?

- In CNN, the receptive field of a neuron (or feature) refers to the **region of the input image (or signal) that influences that neuron's activation**.
- The receptive field is the **portion of the input** that the neuron "looks at" or "receives information from."

For a single 1D convolution layer with kernel size k, dilation d, and stride $s = 1$, the size of the receptive field R can be calculated as:

$$R = (k - 1) \cdot d + 1$$

This equation shows that as the dilation rate increases, the receptive field expands without increasing the number of learnable parameters.

Example 1D Kernel size : 3

Dilation = 1:



Dilation = 2:



Dilation = 3:



Let's say we have a 1D kernel of size 3:

- dilation=1 → **receptive field size** $= (3 - 1) \times 1 + 1 = 3$
- dilation=2 → **receptive field size** $= (3 - 1) \times 2 + 1 = 5$
- dilation=3 → **receptive field size** $= (3 - 1) \times 3 + 1 = 7$

nn.Conv2d in PyTorch

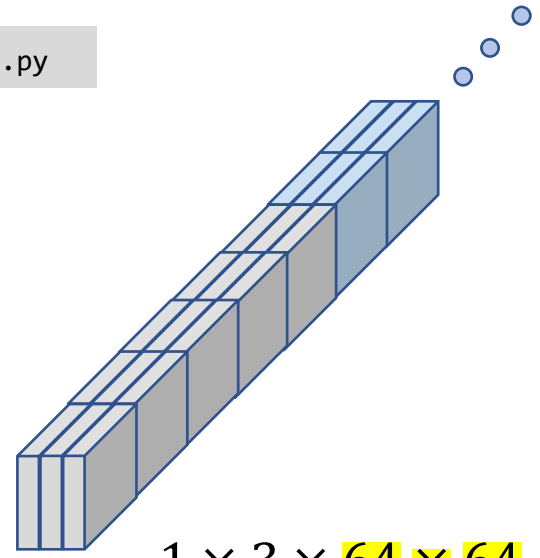
simple_conv.py

```
import torch
import torch.nn as nn

# 2D Conv ================================================
x = torch.randn(1, 3, 64, 64)  # batch=1, RGB image 64x64

conv2d = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride=1,
padding=1)
y = conv2d(x)
print(y.shape)  # [1, 16, 64, 64]
```

```
torch.Size([1, 16, 64, 64])
```



$1 \times 3 \times 64 \times 64$

(batch, channel, height, width)

nn.Conv3d in PyTorch

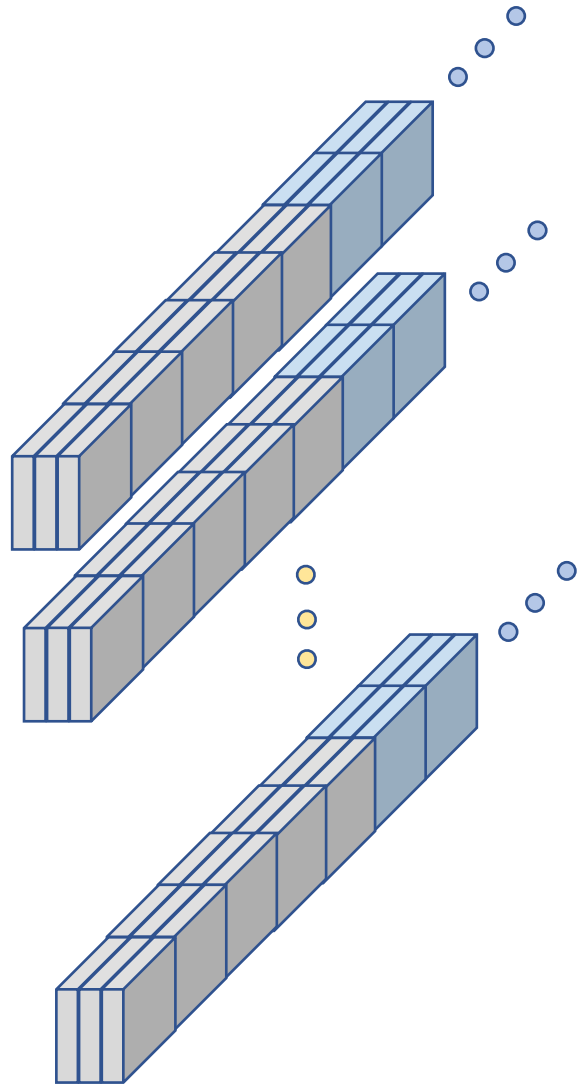
simple_conv.py

```
import torch
import torch.nn as nn

# 3D Conv ================================================
x = torch.randn(1, 3, 16, 64, 64)  # 16 frames of 64x64 RGB video

conv3d = nn.Conv3d(in_channels=3, out_channels=8, kernel_size=3, stride=1, padding=1)
y = conv3d(x)
print(y.shape)  # [1, 8, 16, 64, 64]
```

```
torch.Size([1, 8, 16, 64, 64])
```



$$1 \times 3 \times 16 \times 64 \times 64$$

(batch, channel, depth, height, width)

Pooling

Filter size: 2×2

Stride: 2×2

2	3	2	0
5	-2	2	8
-1	-6	7	3
-4	-5	4	2

Feature map

Max Pooling (Modern, e.g., after AlexNet (2012))



5	8
-1	7

- Keeps "**strongest activation**"
- Emphasizes **selectivity**
- Handles complex data and focuses on discriminative power

Average Pooling (Old, e.g., LeNet-5 (1998))



2	3
-4	4

- "**Smooths** out variations"
- Emphasizes **stability**
- Handles simple data and focuses on invariance

Pooling

- In Convolutional Neural Networks (CNNs), pooling layers are used to **reduce the spatial dimensions (width and height) of feature maps** while **preserving the most important information.**

- This helps make the network more **computationally efficient** and **less sensitive to small translations or distortions in the input image**.

- Max Pooling (after AlexNet (2012))
- Average Pooling (Old, e.g., LeNet-5 (1998))

Max Pooling

It captures the **most prominent feature** (such as edges or textures) within that region.

Advantages:

- Provides **translation invariance**.
- Helps the model focus on **strongest activations**.
- **Reduces overfitting by reducing spatial size**.
- It added **nonlinearity** and improved training efficiency.

Average Pooling

It retains **overall smooth information** rather than emphasizing a single strong activation.

Advantages:

- Maintains more background or contextual information.
- Produces **smoother** and less noisy representations.

Max Pooling vs. Average Pooling in PyTorch

```python
import torch
import torch.nn as nn

# Create a simple 4x4 input tensor (1 channel, batch size = 1)
x = torch.tensor([[
    [[2.0, 3.0, 2.0, 0.0],
     [5.0, -2.0, 2.0, 8.0],
     [-1.0, -6.0, 7.0, 3.0],
     [-4.0, -5.0, 4.0, 2.0]]
]])

# Define pooling layers
max_pool = nn.MaxPool2d(kernel_size=2, stride=2)
avg_pool = nn.AvgPool2d(kernel_size=2, stride=2)

# Apply pooling
max_out = max_pool(x)
avg_out = avg_pool(x)

print("\nMax Pooling Output:")
print(max_out)

print("\nAverage Pooling Output:")
print(avg_out)

print(x.shape, max_out.shape) # [ batch size, number of channels, height, width ]
```

```
Max Pooling Output:
tensor([[[[ 5.,  8.],
          [-1.,  7.]]]])

Average Pooling Output:
tensor([[[[ 2.,  3.],
          [-4.,  4.]]]])
torch.Size([1, 1, 4, 4]) torch.Size([1, 1, 2, 2])
```

simple_pooling.py

Pooling layers in PyTorch

- The `torch.nn` module includes pooling layers for 1D, 2D, and 3D data:

Layer	Typical Input	Use Case	Example **Input Shape**
`nn.MaxPool1d`	1D signals	Audio, text (sequences), or 1D feature maps	[batch, channels, length]
`nn.MaxPool2d`	2D images	Image processing (CNNs)	[batch, channels, height, width]
`nn.MaxPool3d`	3D volumes	Video, medical imaging (e.g., MRI)	[batch, channels, depth, height, width]

nn.MaxPool1d in PyTorch

```python
import torch
import torch.nn as nn

# 1D Max Pooling ================================================
x = torch.randn(1, 1, 8)  # [batch=1, channel=1, length=8]

pool1d = nn.MaxPool1d(kernel_size=2, stride=2)
print(pool1d(x).shape)  #  [1, 1, 4]
```

pooling_layers.py

```
torch.Size([1, 1, 4])
```

```python
torch.nn.MaxPool1d(
    kernel_size,    # Size of the pooling window (int)
    stride=None,    # Step size (default = kernel_size)
    padding=0,      # Add zeros before/after sequence (rarely used)
    dilation=1,     # Spacing between elements in window (usually 1)
    return_indices=False, # If True, also return the index of max value
    ceil_mode=False # If True, use ceil() instead of floor() for output size
)
```

nn.MaxPool2d and nn.MaxPool3d in PyTorch

```python
import torch
import torch.nn as nn

# 2D Max Pooling ================================================
x = torch.randn(1, 1, 8, 4) # [batch, channel, height, width]

pool2d = nn.MaxPool2d(kernel_size=2, stride=2)
print(pool2d(x).shape)  #  [1, 1, 4, 2]

# 3D Max Pooling ================================================
x = torch.randn(1, 1, 4, 8, 4)  # [batch, channel, depth, height, width]

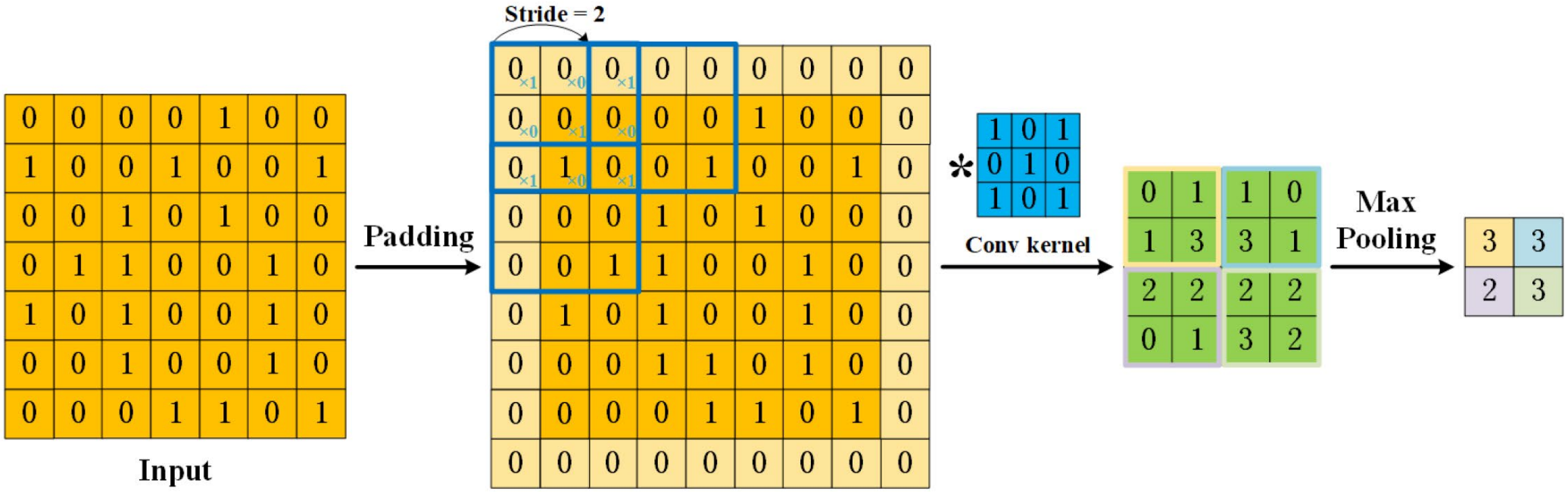
pool3d = nn.MaxPool3d(kernel_size=2, stride=2)
print(pool3d(x).shape)   #  [1, 1, 2, 4, 2]
```

```
torch.Size([1, 1, 4, 2])
torch.Size([1, 1, 2, 4, 2])
```

pooling_layers.py

Pipeline of simple 2D CNN

Example



Max Pooling vs. Average Pooling in PyTorch

```python
import torch
import torch.nn as nn

# Create a simple 4x4 input tensor (1 channel, batch size = 1)
x = torch.tensor([[
    [[2.0, 3.0, 2.0, 0.0],
     [5.0, -2.0, 2.0, 8.0],
     [-1.0, -6.0, 7.0, 3.0],
     [-4.0, -5.0, 4.0, 2.0]]
]])

# Define pooling layers
max_pool = nn.MaxPool2d(kernel_size=2, stride=2)
avg_pool = nn.AvgPool2d(kernel_size=2, stride=2)

# Apply pooling
max_out = max_pool(x)
avg_out = avg_pool(x)

print("\nMax Pooling Output:")
print(max_out)

print("\nAverage Pooling Output:")
print(avg_out)

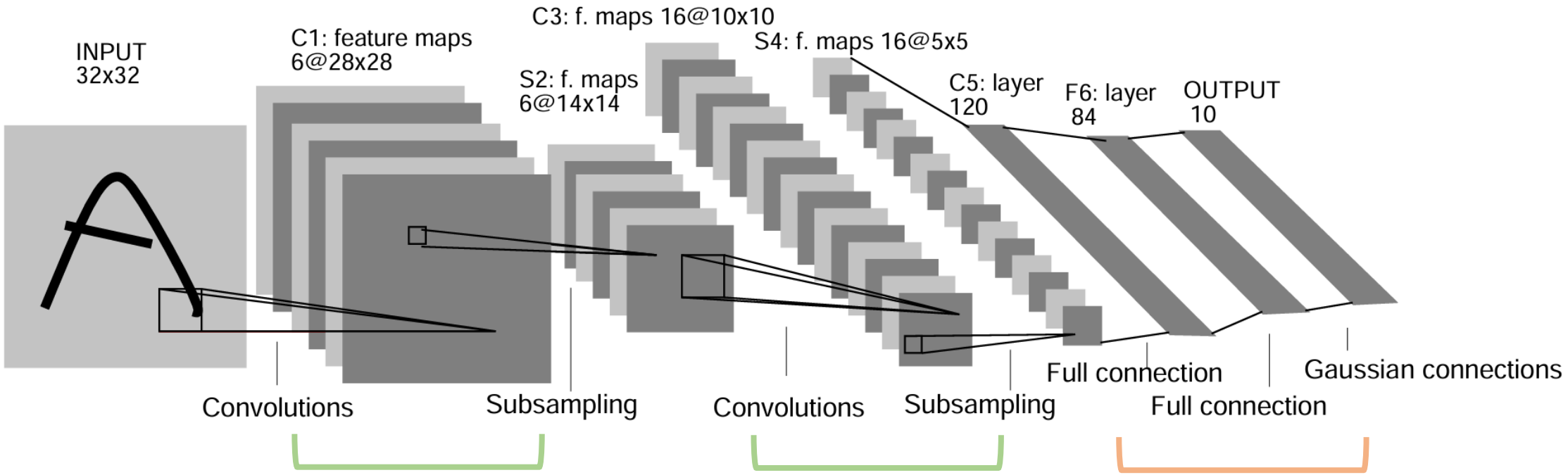
print(x.shape, max_out.shape) # [ batch size, number of channels, height, width ]
```

```
Max Pooling Output:
tensor([[[[ 5.,  8.],
          [-1.,  7.]]]])

Average Pooling Output:
tensor([[[[ 2.,  3.],
          [-4.,  4.]]]])
torch.Size([1, 1, 4, 4]) torch.Size([1, 1, 2, 2])
```

Simple_pooling.py

LeNet-5 (1998)



- "CNN" which is trained end-to-end with **backpropagation** for image tasks (handwritten digit recognition).
- It used convolution + subsampling (old term: = average pooling) + fully connected layers.

AlexNet (2012)

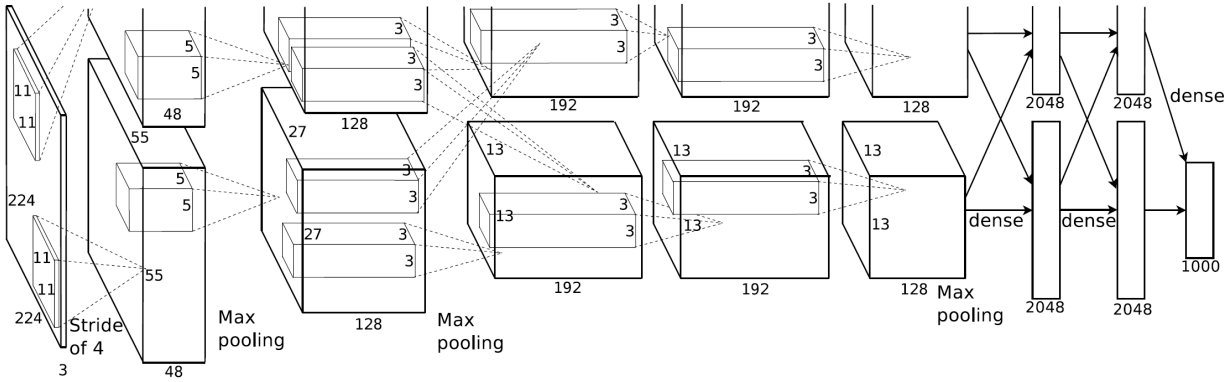


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

Key Innovations

Feature	Description
Deep Architecture	8 layers total (5 convolutional + 3 fully connected).
ReLU Activation	Introduced ReLU for faster and more stable training.
GPU Training	Used two GPUs in parallel for large-scale training.
Data Augmentation	Applied random crops, flips, and color shifts.
Dropout	Used dropout (p=0.5) in fully connected layers to prevent overfitting.
Local Response Normalization (LRN)	Applied normalization inspired by biological neurons.
Overlapping Pooling	Used max pooling with overlapping windows.

AlexNet is a Convolutional Neural Network (CNN) architecture proposed by **Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton** in 2012. It was introduced in the paper titled *"ImageNet Classification with Deep Convolutional Neural Networks"* (NIPS 2012).

The model won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2012 with a top-5 error rate of 15.3%, compared to 26.2% for the next best entry.

AlexNet (2012)

Architecture Overview

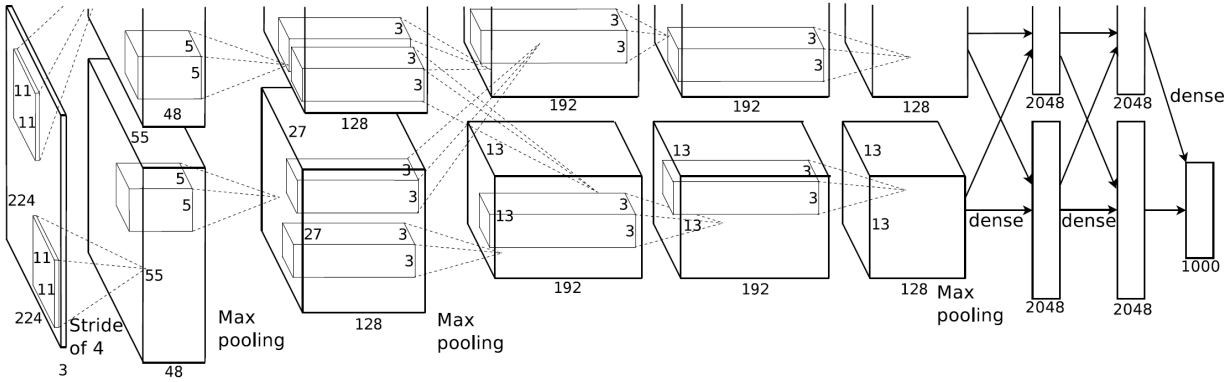


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

Layer	Type	Details
1	Conv	96 filters of size 11x11, stride 4 → ReLU → LRN → Max Pooling
2	Conv	256 filters of size 5x5 → ReLU → LRN → Max Pooling
3	Conv	384 filters of size 3x3 → ReLU
4	Conv	384 filters of size 3x3 → ReLU
5	Conv	256 filters of size 3x3 → ReLU → Max Pooling
6	FC	4096 neurons → ReLU → Dropout
7	FC	4096 neurons → ReLU → Dropout
8	FC	1000 neurons → Softmax output

AlexNet was a turning point in computer vision and deep learning. It proved that deep CNNs could outperform traditional methods such as SIFT and HOG. The model demonstrated the potential of GPU acceleration and inspired later architectures such as VGG, GoogLeNet, and ResNet.

VGG (2014)

VGGNet (Visual Geometry Group Network) is a Convolutional Neural Network (CNN) architecture proposed by **Karen Simonyan and Andrew Zisserman** from the University of Oxford in 2014. It was introduced in the paper titled *"Very Deep Convolutional Networks for Large-Scale Image Recognition"* and became one of the most influential models in computer vision.

The paper systematically investigated the effect of network depth on large-scale image recognition, showing that deeper networks with small convolutional filters perform better on datasets such as ImageNet.

Key Design Principles

Concept	Description
Small Filters	Stacked multiple 3×3 convolutions instead of large kernels (5×5 or 7×7). This increases non-linearity and reduces parameters.
Uniform Architecture	Repeated Conv-ReLU blocks followed by max pooling, creating a simple and consistent structure.
Max Pooling	Used 2×2 pooling with stride 2 to reduce spatial resolution after each block.
Fully Connected Layers	Two FC layers of 4096 neurons followed by a classification layer with 1000 outputs.
ReLU Activation	Applied ReLU after every convolution and fully connected layer.

VGG-16 Architecture Example

Input: 224×224 RGB image

Block	Layers	Details
1	Conv1_1, Conv1_2	64 filters (3×3) → ReLU → MaxPool (2×2)
2	Conv2_1, Conv2_2	128 filters (3×3) → ReLU → MaxPool (2×2)
3	Conv3_1, Conv3_2, Conv3_3	256 filters (3×3) → ReLU → MaxPool (2×2)
4	Conv4_1, Conv4_2, Conv4_3	512 filters (3×3) → ReLU → MaxPool (2×2)
5	Conv5_1, Conv5_2, Conv5_3	512 filters (3×3) → ReLU → MaxPool (2×2)
FC Layers	FC6, FC7, FC8	4096 → 4096 → 1000 (Softmax)

Strengths

- Simple and elegant architecture that is easy to generalize.
- Demonstrated the effectiveness of deep networks with small filters.
- Widely used as a feature extractor for transfer learning.

Limitations

- High computational and memory cost (large number of parameters).
- Lacks residual connections, leading to difficulties training deeper versions.
- Fully connected layers dominate parameter count.

The Optimization Problem

Before ResNet, making networks deeper often led to worse performance, not due to overfitting, but because deeper models were harder to optimize.

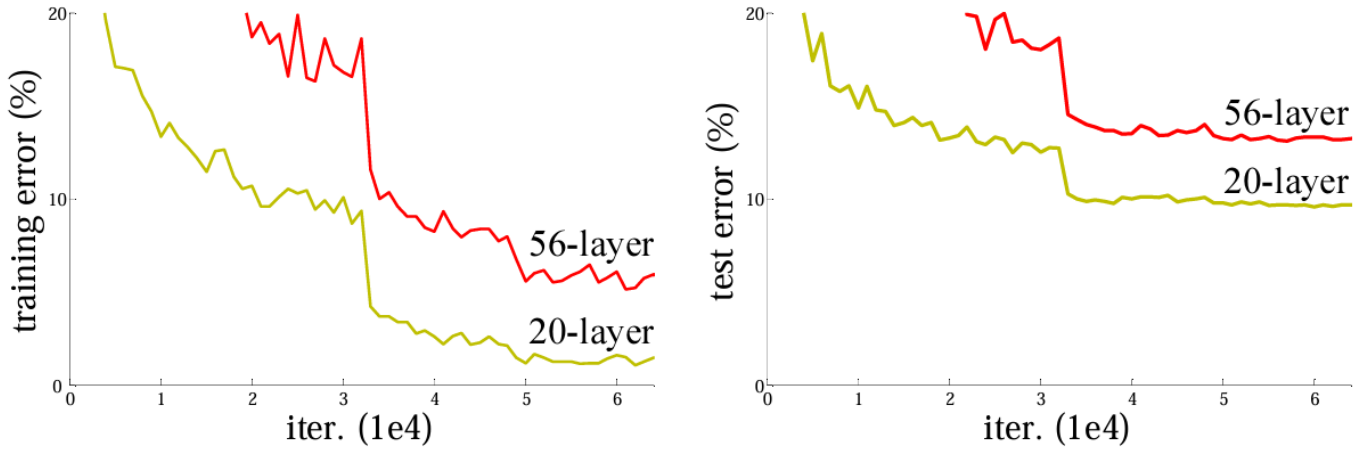
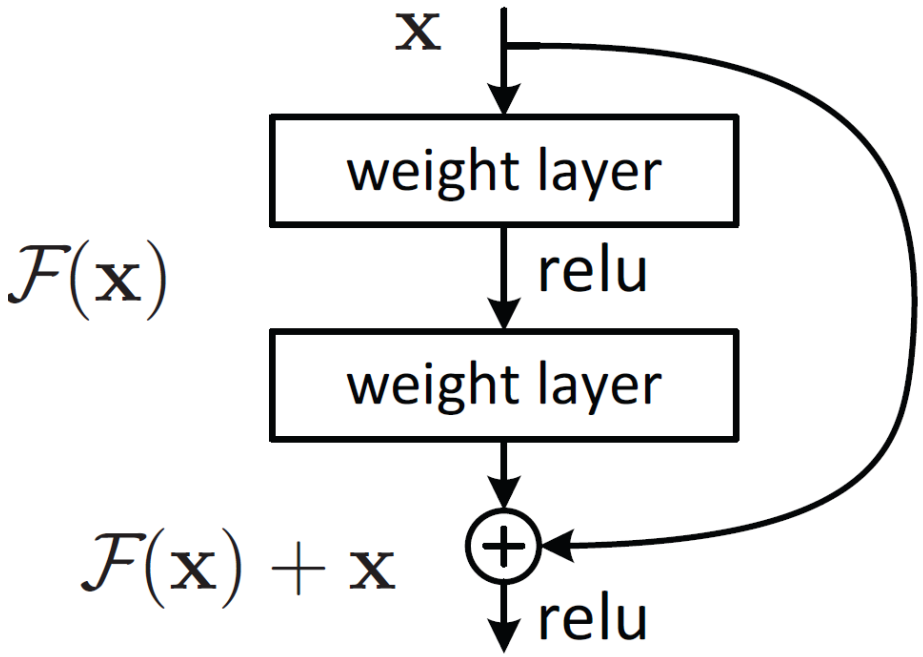


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

ResNet (2016)

ResNet (Residual Network) was introduced by **Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun** in 2015. The work was published as *"Deep Residual Learning for Image Recognition"* in CVPR 2016. The paper is available at arXiv:1512.03385.

ResNet was a breakthrough in deep learning, enabling the training of extremely deep neural networks (up to 152 layers and beyond) without degradation problems. It solved the vanishing and exploding gradient issues that plagued earlier deep architectures.



The Solution: Skip (Residual) Connections

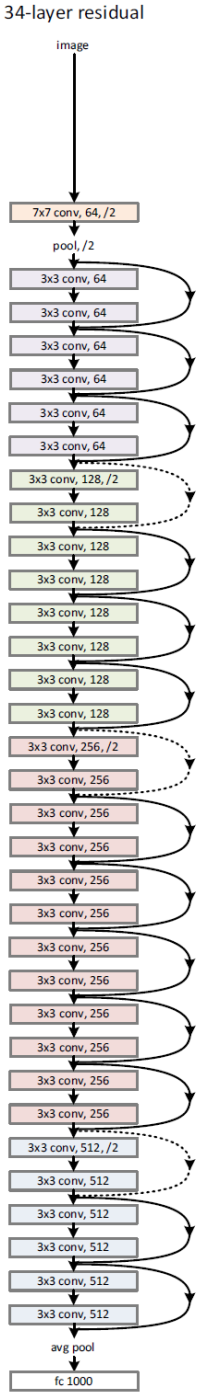
ResNet introduced the concept of residual connections. Instead of learning a direct mapping $H(x)$, the network learns a residual function $F(x)$ such that:

$$H(x) = F(x) + x \tag{1}$$

Here, x is the input, and $F(x)$ is the output of a few stacked layers. The shortcut connection adds the input directly to the output, allowing gradients to flow easily backward during training.

identity

Residual learning: a building block.



layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

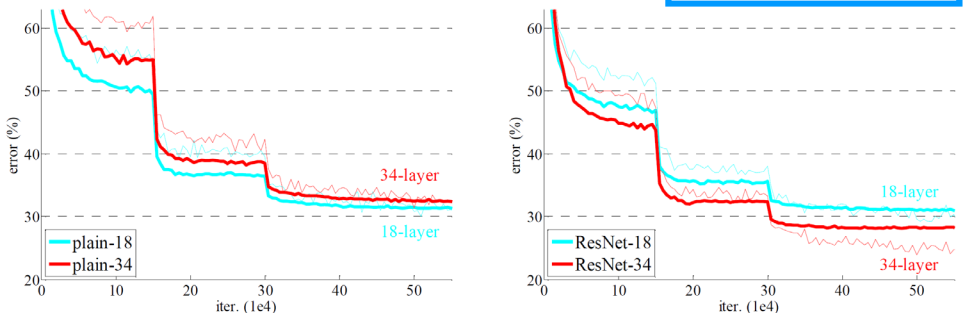


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

Basic Block

A typical residual block contains two convolutional layers with batch normalization and ReLU activation:

```
Input → [Conv → BN → ReLU → Conv → BN] → + (skip connection) → ReLU → Output
```

Bottleneck Block

For deeper networks such as ResNet-50, 101, and 152, a "bottleneck" block is used to reduce computational cost:

```
1×1 Conv → 3×3 Conv → 1×1 Conv → + skip connection
```

Common ResNet Variants

Model	Layers	Structure
ResNet-18	18	Uses basic residual blocks (2 convs per block).
ResNet-34	34	Similar to ResNet-18 but deeper.
ResNet-50	50	Uses bottleneck blocks (1×1, 3×3, 1×1).
ResNet-101	101	Deeper version with more bottleneck blocks.
ResNet-152	152	Very deep network achieving state-of-the-art ImageNet accuracy.

Key Results

- Achieved a top-5 error rate of 3.57% on ImageNet, winning the ILSVRC 2015 competition.
- Enabled networks with more than 1000 layers to be trained effectively.
- Introduced a general building block reused in many later architectures.

Why Residual Connections Work

- Facilitate gradient flow, reducing vanishing gradients.
- Allow the network to learn identity mappings easily if needed.
- Improve convergence and accuracy for very deep models.

Simple CNN (1)

Example 1

simple_cnn_1.py

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()  # Run the initialization code of nn.Module before adding my own layers.

        # 1st convolutional layer: input channels=3 (RGB), output channels=16
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
        # 2nd convolutional layer: input=16, output=32
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)

        # Max pooling layer
        self.pool = nn.MaxPool2d(2, 2)

        # Fully connected layers
        self.fc1 = nn.Linear(32 * 8 * 8, 128)  # CIFAR10 images are 32x32
        self.fc2 = nn.Linear(128, 10)  # 10 classes

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))  # Conv1 → ReLU → Pool
        x = self.pool(F.relu(self.conv2(x)))  # Conv2 → ReLU → Pool
        x = x.view(-1, 32 * 8 * 8)            # Flatten
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = SimpleCNN()
print(model)
```

```
SimpleCNN(
  (conv1): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=2048, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)
```

Simple CNN (1)

Example 1

- Input tensor: $[B, 3, 32, 32]$ (batch of RGB images)
- Output tensor: $[B, 10]$ (class logits)

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()  # Run the initialization code of nn.Module before adding my own layers.

        # 1st convolutional layer: input channels=3 (RGB), output channels=16
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
        # 2nd convolutional layer: input=16, output=32
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)

        # Max pooling layer
        self.pool = nn.MaxPool2d(2, 2)

        # Fully connected layers
        self.fc1 = nn.Linear(32 * 8 * 8, 128)  # CIFAR10 images are 32x32
        self.fc2 = nn.Linear(128, 10)  # 10 classes

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))  # Conv1 → ReLU → Pool
        x = self.pool(F.relu(self.conv2(x)))  # Conv2 → ReLU → Pool
        x = x.view(-1, 32 * 8 * 8)            # Flatten
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = SimpleCNN()
print(model)
```

Stage	Output Shape
Input	$[B, 3, 32, 32]$
Conv1 + ReLU + Pool	$[B, 16, 16, 16]$
Conv2 + ReLU + Pool	$[B, 32, 8, 8]$
Flatten	$[B, 2048]$
FC1 + ReLU	$[B, 128]$
FC2	$[B, 10]$

simple_cnn_1.py

```
SimpleCNN(
  (conv1): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=2048, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)
```

- `torch`: Core PyTorch library.
- `torch.nn`: Provides building blocks for neural networks such as `Linear` and `Conv2d`.
- `torch.nn.functional`: Functional API that provides stateless versions of operations such as `relu()` and `max_pool2d()`.

1. **Conv1 + ReLU + Pool:** Input $[B, 3, 32, 32] \rightarrow$ Output $[B, 16, 16, 16]$.
2. **Conv2 + ReLU + Pool:** Input $[B, 16, 16, 16] \rightarrow$ Output $[B, 32, 8, 8]$.
3. **Flatten:** Reshape into a 1D vector of size $32 \times 8 \times 8 = 2048$ per sample.
4. **FC1 + ReLU:** Linear transformation to 128 hidden units.
5. **FC2:** Final output of size 10 (class logits).

Thank you!

